

11



Project Management

*Take off your engineering hat and put on your management hat.
—Statement made during of the 51-L launch discussion*

Managers do not generate a useful product of their own. Instead, they provide and coordinate resources so that others can generate useful products. Managing a software project requires a combination of managerial and social skills to foresee potentially damaging problems and to implement the appropriate response. Because of the lack of visible product and the reliance on nontechnical skills, management is usually the target of jokes by developers and others. Management, however, is a critical function for bringing a project to a successful end, given the complexity of current systems and the high rate of change during development.

Managers do not make technical decisions and often do not have the background to make such decisions. Instead, they are responsible for coordinating and administrating the project and ensuring that the high-quality system is delivered on time and within budget. The main tools of management are planning, monitoring, risk management, and contingency handling.

In this chapter, we describe the management activities that are visible to developers and the team leader. We assume a two-level management hierarchy, typical of many software industries today. Our goal is not to provide the reader with project management expertise but rather to give sufficient knowledge to be able to act as a team leader and effectively interact with project management. Consequently, we cover topics such as task plans, role assignments, risk management, and software project management plans; we leave out topics such as cost estimation, labor issues, and contract negotiation.

11.1 Introduction: The STS-51L launch decision

Consider the following example:

Example:^a The STS-51L launch decision

On January 28, 1986, the Space Shuttle Challenger flying mission STS-51L exploded 73 seconds into flight, killing all seven crew members aboard. The Rogers Presidential Commission, formed to investigate the accident, determined that combustion gases leaking through a joint in the right solid rocket booster caused the ignition of hydrogen fuel from the external fuel tank. The Rogers Commission also determined that there were serious flaws in the decision-making process at NASA.

The main components of the Space Shuttle are the orbiter, the solid rocket boosters, and the external tank. The orbiter is the vehicle that transports cargo and astronauts. The solid rocket boosters provide most of the thrust necessary to put the orbiter in orbit. When they consume all their fuel, they are detached and fall back into the ocean. The external tank, providing fuel to the orbiter's engine, is also detached before reaching orbit. The solid booster rockets were built by a private contractor, Morton Thiokol. The solid booster rockets were built-in sections such that they could be easily transported. Once delivered to the launch site, the sections are assembled. Each section is connected by a joint. A design flaw of this joint was responsible for the combustion gas leak that caused the accident.

The Rogers Commission determined that the engineers at Thiokol, the contractor who built the solid rocket boosters, were aware of possible failure of the solid booster rocket joints. Solid booster rockets recovered from previous shuttle flights indicated that rubber gaskets, called O-rings, in the field joints had been eroded by combustion gases. This erosion seemed particularly severe on flights that took place in cold temperatures. This erosion concerned engineers, because the O-rings had not been designed to come in contact with the combustion gases. Managers treated the joint problem as an acceptable risk, given that erosion had occurred in previous missions without causing an incident. Engineers at Thiokol had repeatedly and unsuccessfully attempted to communicate the severity of the problem to their management. In the days preceding the STS-51L launch, engineers were alarmed by the low temperatures at the launch site and attempted to convince their management to recommend against a launch. Thiokol initially recommended against the launch and then later reversed its position, at the urging of NASA.

Both NASA and Thiokol have hierarchical reporting structures, typical of public administration organizations. Moreover, the information flow through the organization is through the reporting structure. Engineers report to their group manager who then reports to the project manager and so on. Each level can choose to modify, distort, or suppress any information before sending it upward, based on its own objectives. The STS-51L mission had been delayed repeatedly and was already several months late. Management at both NASA and Thiokol wanted to meet the deadline. This situation resulted in a lack of communication, improper information dissemination, and inadequate risk evaluation at both NASA and Thiokol.

The Rogers Commission determined that the primary cause of the Challenger accident was a design flaw in the solid rocket booster joint. The Rogers Commission, however, also determined that the decision-making process and the communication structure at Thiokol and NASA were flawed and failed to assess the risks of launching the flawed vehicle.

- a. The Challenger accident has been summarized here as an example of project management failure. Both the mechanical and the administrative failures that caused the Challenger accident are much more complex than this summary suggests. For complete reference of the investigation, refer to the Rogers Commission report [Rogers et al., 1986] and related books on the subject [Vaughan, 1996].

Project management ensures the delivery of a quality system on time and within budget. The main components of this definition are quality, time, and money. Ensuring delivery within budget requires a manager to estimate and assign the resources required by the project in terms of participants, training, and tools. Ensuring on-time delivery requires a manager to plan work effort and to monitor the status of the project in terms of tasks and work products. Finally, ensuring quality requires a project manager to provide problem reporting mechanisms and monitor the status the product in terms of defects and risks. All three aspects of the project, quality, budget, and time, are essential and need to be addressed together for the project to succeed.

In this chapter, we describe the role of management in software projects. However, the material presented here is far from sufficient for learning to act as a project manager, on the Shuttle program or otherwise. This chapter is oriented toward developers who will become team leaders after some project experience. In this chapter, we assume a two-level management hierarchy, with the project manager at the top, the team leader at the middle, and developers at the bottom. This is large enough for experiencing the effects of complexity on management and small enough to be typical of today's software industry.

In Section 11.2, we provide a bird's eye view of management and its relationship to other development activities. In Section 11.3, we describe the models used for management, such as organizations, roles, work products, tasks, and schedules. In Section 11.4, we describe the management activities that manipulate these models and illustrate how these building blocks can be used together using an example from building facilities. In Section 11.5, we describe management issues related to project management.

11.2 An overview of project management

Project management is concerned with planning and allocating resources to ensure the delivery of quality software system on time and within budget. Project management is subject to the same barriers as technical activities: complexity and change. Complex products require a large number of participants with diverse skills and backgrounds. Competitive markets and evolving requirements introduce change in the development, triggering frequent resource reallocation and making it difficult to track the status of the project. Managers deal with complexity and change the same way developers do: through modeling, communication, rationale, and configuration management. We have talked about the communication, rationale, and configuration aspects of management in previous chapters (Chapters 3, 8, and 10, respectively). In this chapter, we focus on the management models and the activities needed to create and maintain them.

Management models allow to represent the resources available to the project, the constraints resources are subjected to, and the relationships among resources. In this chapter, we describe the following elements:

- **Teams** represent sets of participants who work on a common problem.
- **Roles** represent sets of responsibilities. Roles are used to distribute work to participants within a team.
- **Work products** represent the deliverables and intermediate products of the project. Work products are the visible results of work.
- **Tasks** represent work in terms of sequential steps necessary to generate one or more work products.
- **Schedules** represent the mapping of a task model onto a time line. A schedule represents work in terms of calendar time.

The project manager and team leaders construct and maintain these model elements throughout development. The management activities that concern a team leader are outlined in Figure 11-1.

Using these models, team leaders can communicate with managers and developers about status and responsibilities. If a substantial deviation occurs between the planned work and the actual work, the project manager reallocates resources or changes the project environment.

At the highest level of abstraction, the development life cycle can be decomposed into three major phases: **project initiation**, during which the project scope and resources are defined, **steady state**, during which the bulk of development work occurs, and **project termination**, during which the system is delivered and accepted.

Project initiation. During this phase, the project manager defines the scope of the system with the client and constructs the initial version of the management models. Initially, only the project manager, selected team leaders, and the client are involved. The project manager sets up the project environment, organizes teams, hires participants, and kicks off the project. Project initiation includes the following activities:

- **Problem statement definition.** During this activity, the client and the project manager define the scope of the system in terms of functionality, constraints, and deliverables. The client and the project manager agree on acceptance criteria and target dates.
- **Initial top-level design.** The project manager and the system architect decompose the system into subsystems for the purpose of team assignment. The project manager defines teams and work products in terms of subsystems. The system architect and the developers will revise the subsystem decomposition during system design.
- **Team formation.** The project manager assigns a team to each subsystem, defines cross-functional teams, and selects team leaders. The project manager and the team leaders together then assign roles and responsibilities to participants as a function of the skills of the participants. The team leaders establish training needs for the team during this phase.
- **Communication infrastructure setup.** The project manager and the team leaders set up the communication infrastructure of the project, including bulletin boards, Web sites,

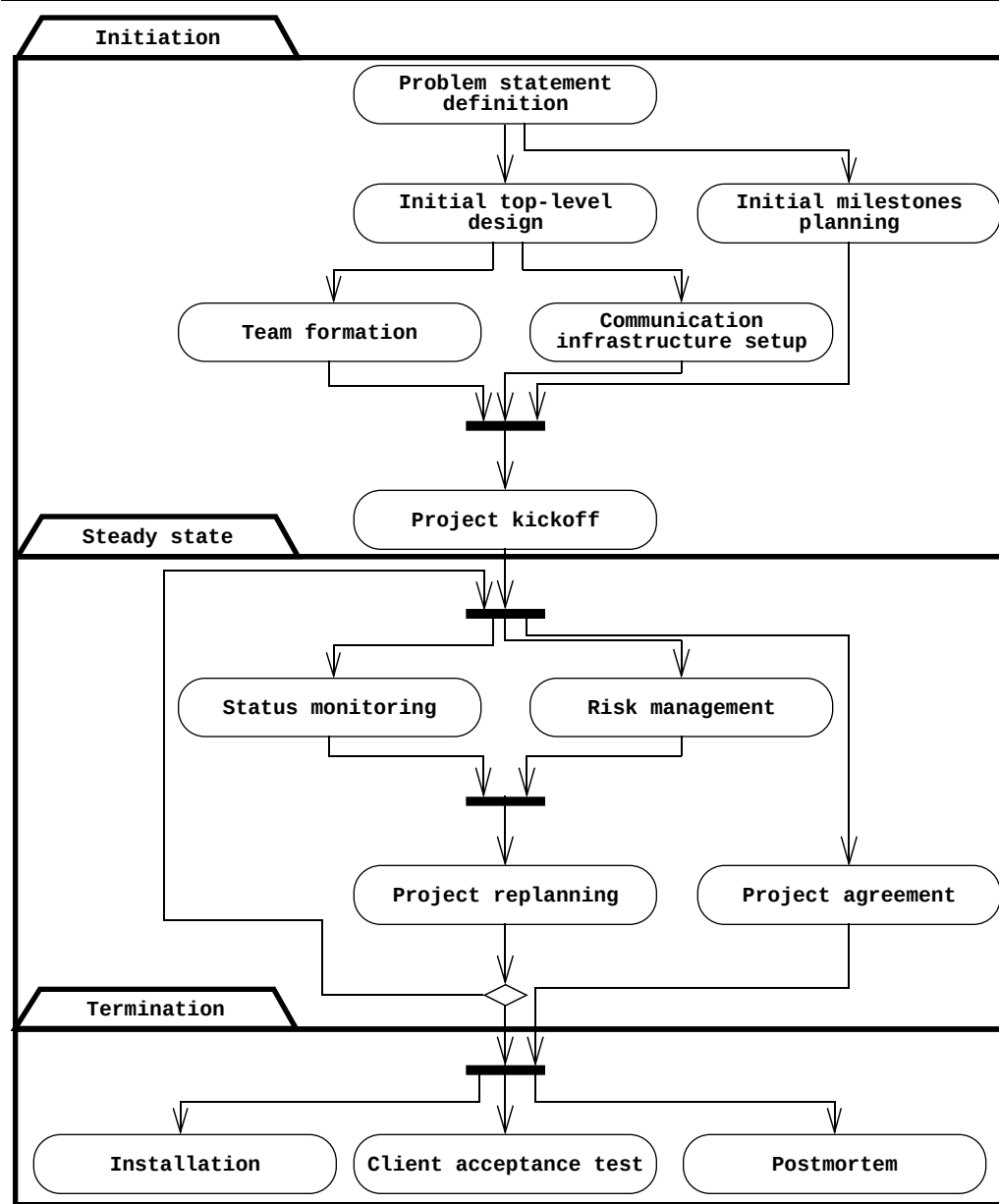


Figure 11-1 Management activities during initiation, steady state, and termination. (UML activity diagram).

configuration management tools, document templates, and meeting procedures. Project participants will use this infrastructure for distributing information and report problems in a timely manner.

- **Initial milestones planning.** The project manager schedules intermediate work products and external deliverables and assigns them to teams.
- **Project kickoff.** The project manager, the team leaders, and the client officially start the project. The purpose of the kickoff meetings is to explain to the project participants the scope of the project, the communication infrastructure, and the responsibilities of each team. After the kickoff, the project is in steady state.

Steady state. During this phase, the role of team leader becomes critical. During project initiation, most decisions are made by the project manager. During steady state, team leaders are responsible for tracking the status of their team and identifying problems via team meetings. Team leaders report the status of their team to the project manager who then evaluates the status of the complete project. Team leaders respond to deviation from the plan by reallocating tasks to developers or obtaining additional resources from the project manager. The project manager is still responsible for the interaction with the client, obtaining formal agreement and renegotiating resources and deadlines. Steady state activities include the following:

- **Project agreement definition.** Once the analysis model is stable, the client and the project manager formally agree on the scope of the system and the delivery date. The *Project Agreement* document is derived from the problem statement and revised during analysis.
- **Status monitoring.** Throughout the project, team leaders and management monitor the status and compare it with the planned schedule. The team leaders are responsible for collecting status information through meetings, reviews, problem reports, and work product completion, and for summarizing it for the project manager.
- **Risk management.** During this activity, project participants identify the potential problems that could cause delays in the schedule and budget overruns. The project manager and the team leaders identify, analyze, and prioritize risks and prepare contingency plans for the important risks.
- **Project replanning.** When the project deviates from the schedule or when a contingency plan is activated, the project manager or the team leader revises the schedule and reallocates resources to meet the delivery deadline. The project manager can hire new staff, create new teams, or merge existing teams.

Project termination. During this phase, the product is delivered and the project history is collected. Most of the developers' involvement with the project ends before this phase. A handful of key developers, the technical writers, and the team leaders are involved with wrapping up the system for installation and acceptance and collecting the project history for future use.

- **Client acceptance test.** The system is evaluated with the client according to the acceptance criteria set forth in the *Project Agreement*. Functional and nonfunctional requirements are demonstrated and tested using scenarios defined in the *Project Agreement*. The client formally accepts the product at this stage.
- **Installation.** The system is deployed in the target environment and documents are delivered. The installation may include user training and a roll out phase, during which data is moved from the previous system to the new system. Acceptance usually precedes installation.
- **Postmortem.** The project manager and the team leaders collect the history of the project for the organizational learning. By capturing the history of major and minor failures and by analyzing their causes, an organization can avoid repeating mistakes and improve its practice.

In all the above activities, timely and accurate communication between project management, the team leaders, and the developers is crucial for determining the status of a project and for making correct decisions. Several tools are available to the project participants, including the communication infrastructure, management documents, such as the *Software Project Management Plan* (SPMP), and, more generally, encouraging the free exchange of information. A team leader can substantially improve communication by rewarding accurate reporting instead of punishing the bearer of bad news, by releasing as much information as possible in a timely manner, and by responding constructively to problem reports.

Managers use models to deal with complexity and revise them to deal with change. In the next section, we describe the model elements used for management and represent them using UML class diagrams. In Section 11.4, we describe the activities during which these models are created and revised using object diagrams and activity diagrams.

11.3 Management concepts

In this section, we describe the main concepts used during project management. Each element represents work from a different perspective (Figure 11-2).

- A **team** is a small group of people who work on the same subproblem (Section 11.3.1). The concept of team deals with the complexity associated with communicating among many persons. Instead of organizing and coordinating project participants as a large group, the project is divided into teams, each tackling a separate subproblem.
- A **role** represents a set of responsibilities assigned to a participant or a team (Section 11.3.2). The clear definition of roles enables unplanned tasks to be easily discovered and assigned. The role of a tester, for example, is to design and execute tests. If new unplanned testing activities are identified, they are assigned to the participant filling the tester role.

- A **work product** represents a concrete deliverable produced by a role (Section 11.3.3). Work products include technical and managerial documents, product features, test cases, test results, technology surveys, and usability reports.
- A **task** represents a work product in terms of an activity (Section 11.3.4). A task results in one or more work products, is assigned to a role, and consumes resources. Tasks are related to each other by dependencies. For example, *Database test planning* is the task of designing tests for the database subsystem. It requires the interface to the subsystem and results into a test plan and a test suite to be executed during the *Database test task*.
- A **schedule** represents tasks assigned to a time line (Section 11.3.5). The schedule is usually the most visible and evolving of all management models. The schedule is revised when estimates are refined, when constraints change, or when resources are reallocated.

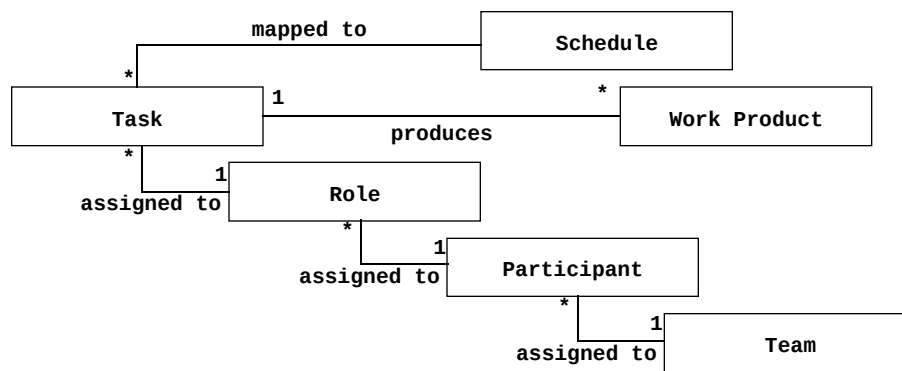


Figure 11-2 Relationships among participants, teams, roles, tasks, work products, and schedules (UML class diagram). Participant are assigned to Teams each responsible for a different subsystem. Each Participant is assigned a number of Roles each corresponding to a set of Tasks. Tasks result in one or more Work Products. A Schedule consists of a set of tasks mapped to a time line.

11.3.1 Teams

A **team** is a small group of people who work on the same subproblem. Instead of organizing and coordinating project participants as a large group of individuals, each cutting away the same problem, the system is divided into subsystems that are each assigned to a team. Each team is responsible for the analysis, design, and implementation of its subsystem. Each team negotiates with other teams the interfaces of the subsystems it needs. Such teams are called **subsystem teams**. Figure 11-3 represents a system architecture of three subsystems mapped to an organization of three subsystem teams.

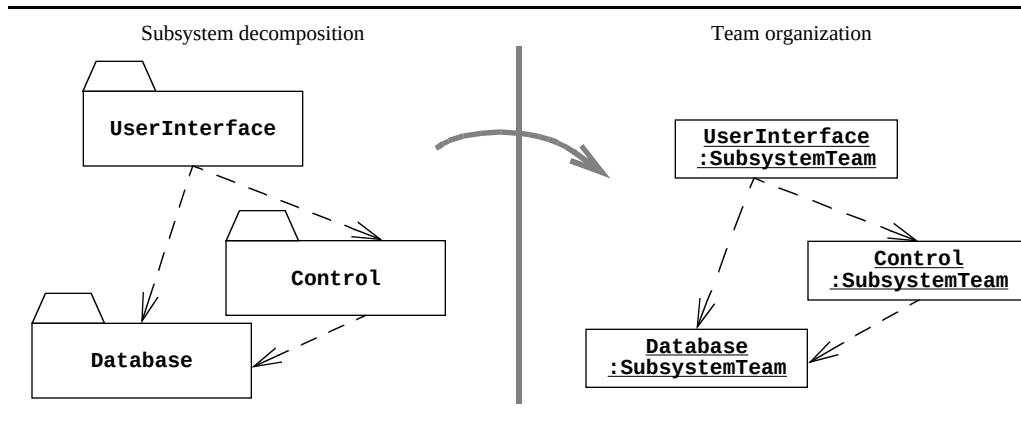


Figure 11-3 An example of a subsystem decomposition mapped onto a team organization (UML object diagram). The arrows represent dependencies among subsystems and among teams.

Each team is managed by a **team leader** who assigns responsibilities to its members, facilitates team meetings, and tracks status. The team leader is often an experienced developer who is familiar with the technical content of the subsystem. The team leader collects and summarizes status information for the project manager and reports open issues.

The hierarchical relationship between developers, team leaders, and the project manager represents the **reporting structure**. The developer, who has the most detailed knowledge of technical details, makes decisions locally and reports to the team leader. The team leader, who has an overview of the subsystem, can override decisions from the developers when necessary and reports to the project manager. The project manager, who has a global view of the objectives of the project and its status, can virtually override any decision. Decisions and guidelines from the project manager are implemented by the team leaders by assigning action items to specific developers. Figure 11-4 illustrates the information flow up and down the reporting structure of a three-subsystem-team organization.

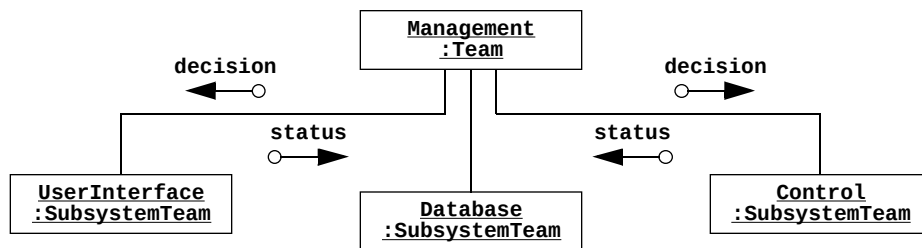


Figure 11-4 Example of reporting structure (UML collaboration diagram). Status information is reported to the project manager and corrective decisions are communicated back to the teams by the team leaders. The team leaders and the project manager are called the management team.

The reporting structure corresponds to the flow of status information and decision information. The reporting relationships, however, represent only a small part of the information flow in the project. Team leaders, for example, usually do not discuss details of the subsystem interface with the project manager. They need, however, to negotiate these details with the teams that depend on their subsystem. Sometimes, team leaders from dependent teams interact directly. Most often, however, a developer, called a **liaison**, is responsible for the task of communicating with another team and shuttling information back and forth. The documentation team, for example, has a liaison to the user interface team to facilitate information about recent changes made to appearance of the system. Teams that do not work directly on a subsystem, but rather, accomplish a project-wide function, are called **cross-functional teams**. Examples of cross-functional teams include the documentation team, the architecture team, and the testing team.

In every organization, there are three types of associations between teams:

- the reporting association, which is used for reporting status information
- the decision association, which is used for propagating decisions
- the communication association, which is used for exchanging other information needed for decisions (e.g., requirements, design models, issues)

In hierarchical organizations, such as a military or church organization, the reporting structure also serves for decision making and communication. In complex software projects, however, most technical decisions are made locally by developers. This requires much information to be exchanged and updated among the teams. This information cannot be exchanged via the reporting structure, given its complexity and volume. Instead, this information exchange is addressed by cross-functional teams and by liaisons. Figure 11-5 depicts an example of team organization with liaisons to cross-functional teams.

Figure 11-6 is an example of team organization for a system with three subsystems. The links between each team and the management team (displayed in thick lines for clarity) represent the reporting structure. All other associations represent communication. The links among the subsystem teams represent communication resulting from dependencies between subsystems.

The role of a team leader is to make sure that not only is the project manager aware of the status of the team, but also that team members have all the information they need from other teams. This requires the selection effective communicators as liaisons and to interact with other team leaders to ensure all necessary communication paths exist.

Other team organizations

In team-based projects, the organization changes over time. If new subsystems are identified in the software architecture, new subsystem teams are formed. If new cross functions are required, liaisons from the subsystem teams are selected and assembled into a new cross-functional teams. Similarly, teams are removed from the organization once their work is

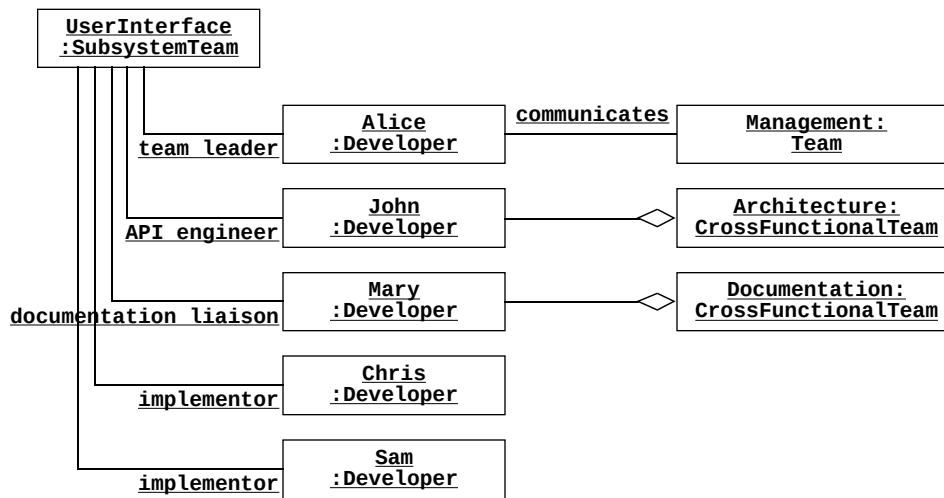


Figure 11-5 Examples of a team organization (UML object diagram). The team is composed of five developers. Alice is the team leader, also called the liaison to the management team. John is the API engineer, also called the liaison to the architecture team. Mary is the liaison to the documentation team. Chris and Sam are implementers and interact with other teams only informally.

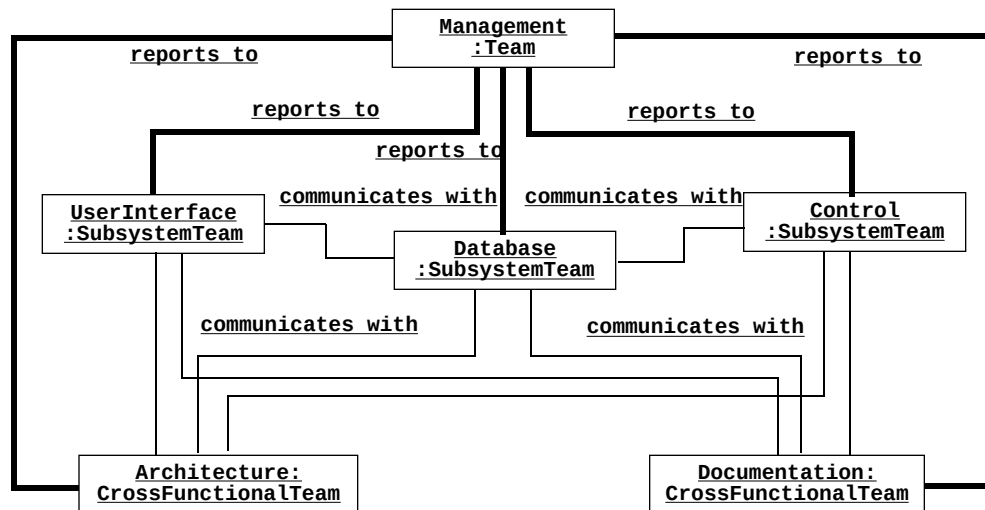


Figure 11-6 An example of a team-based organization with four subsystem teams and three cross-functional teams (UML object diagram). The links between all teams and the management team (drawn thick for clarity) represent the reporting structure. All other associations represent communication.

completed. The team model should be treated the same way as the system model: It is updated to reflect changes in the reality and constantly evolves according to the need of the system and of the project.

Software engineering introduces a trade-off in team organization. Developers have the most technical knowledge about the system and are best able to make decisions locally. However, they optimize only local criteria, which may result in suboptimal decisions at the project level, especially when interteam coordination is difficult. In addition to a team-based organization, different alternative team organizations have been proposed in the literature.

The **chief programmer organization** [Brooks, 1975] is a hierarchical organization that assigns all team decisions to the team leader, called the **chief programmer**. The chief programmer is responsible for the design and development of the subsystem and delegates programming tasks to a pool of specialists. The chief programmer is supported by an assistant chief programmer, who substitutes the chief when needed, and a librarian, responsible for documentation and all clerical functions. The chief programmer has the final say on everything.

Whereas the chief programmer organization takes a hierarchical position on management, **egoless programming** [Weinberg, 1971] takes a democratic one. Responsibility is assigned to the team instead of to the individuals. Results, not participants, are criticized. Issues are resolved by voting. The structure of the team is informal, and communication occurs spontaneously.

In general, there is a spectrum of organizations ranging from a strict hierarchy, such as a military organization, to a dynamic set of collaborating groups. Raymond [Raymond, 1998] refers to the former as the **cathedral model** and the latter as the **bazaar model**. Until recently, it was assumed that the bazaar model was only appropriate for simple systems developed by a team of experienced programmers. It was commonly thought that the cathedral model was necessary for dealing with system complexity. The Linux¹ operating system has provided a counterexample. Linux has been developed from scratch by thousands of developers, most of whom have never met face to face. Moreover, most of these developers are part-time contributors who are not receiving any financial reward for contributing code to Linux.

The complexity of the application domain, the experience of the developers and management, their location, and their communication infrastructure eventually determines where the team organization is located in the cathedral/bazaar spectrum. We might often find that a project's organization starts near the cathedral end of the spectrum and moves toward the bazaar end, once developers gain experience, get to know each other, and develop and communicate a shared model of the system.

Corporate organizations

The emphasis of this book is on software development projects and, consequently, we limit the scope of our discussion to roles up to and including the project manager. We need to

1. Linux is a freely available Unix operation system created by Linus Torwalds. For more information check <http://www.linux.org/>.

recognize that every project occurs in the context of a broader organization, such as a corporation or a university. A project is formed by selecting among the skills, experience, and tools available within the broader organization.

In the **functional organization**, participants are grouped by specialties into departments. A department of analysts constructs the analysis model, a department of designers transform it into a design model, quality assurance is handled by another department, and so on. The functional organization usually corresponds to a sequential order of the life cycle activities, that is, a waterfall model. Projects are pipelined through functional organizations, resulting in few participants being involved for the complete duration of the project. This type of organization is appropriate only for well-defined problems in which the rate of change is low.

The **project-based organization** is an organization in which participants are assigned to only one project at a time for the duration of the project. Participants are usually grouped by subsystems. The information flow between teams, however, is still low, as there are no cross-functional teams or liaisons. Communication and reporting are both done via the reporting structure. The project-based organization is better able to deal with change than the functional organization. The team-based organization we described is a refinement of the project-based organization, in which communication among teams is handled by cross-functional teams. A virtual company is a project-based organization. A set of heterogeneous groups is assembled under one umbrella for the duration of a single project.

The **matrix organization** is a combination of functional and project-based organization. Each participant belongs to a department and is assigned to a project. When the participant's involvement with the project ends, the participant is reassigned to another project. The matrix organization results in two overlapping reporting structures: The participant reports to the department head and to the project manager, with the department head taking precedence. Matrix organizations suffer from the "double-boss" problem when department interests and project interests conflict.

11.3.2 Roles

Once the project is organized into teams, we need to define how team members are assigned individual responsibilities. In this section, we define how to assign a set responsibilities by assigning a **role**. A role defines the types of technical and managerial tasks that are expected from a person. For example, the role of tester of a subsystem team carries the responsibility of defining test suites for the subsystem under development, for executing these tests, and for reporting discovered defects back to the developers.

We distinguish between five types of roles:

- **Management roles** are concerned with the organization and execution of the project within constraints. These roles include the project manager and the team leader.

- **Development roles** are concerned with specifying, designing, and constructing subsystems. These roles include the analyst, the system architect, the object designer, and the implementor.
- **Cross-functional roles** are concerned with coordinating more than one team.
- **Consultant roles** are concerned with providing temporary support in areas where the project participants lack expertise. The users and the client act in most projects as consultants on the problem domain. Technical consultants may also bring expertise on new technologies or methods.
- **Promoter roles** are concerned with promoting changes through an organization. Once the organization is set up, the task plan in place, and the project is running in steady state, it becomes increasingly difficult to introduce changes in the project. Promoters are roles for overcoming barriers to change.

We described the management roles in the previous section. We described development roles in Chapters 5, 6, and 7. Below, we describe in more detail cross-functional roles, consultant roles, and promoter roles.

Cross-functional roles

Cross-functional roles are concerned with coordination among teams. Developers filling these roles are responsible for exchanging information relevant to other teams and negotiating interface details.

The **liaison** is responsible for disseminating information from one team to another. In some cases (such as the API engineer), a liaison functions as a representative of a subsystem team and may be called to resolve interteam issues. There are six types of cross-functional roles:

- The **API engineer** is responsible for the interface definition of the assigned subsystem. The interface has to reflect the functionality already assigned to the subsystem and to accommodate the needs of the dependent subsystems. When functionality is traded off with other subsystems, resulting in subsystem changes, the API engineer is responsible for propagating changes back to the subsystem team.
- The **document editor** is responsible for integrating documents produced by a team. A document editor can be seen as a service provider to other teams that depend on a given subsystem team. A document editor also manages information released internally to the team, such as the meeting agendas and minutes.
- The **configuration manager** is responsible for managing different versions of documents, models, and code produced by a team. For simple configuration management policies (e.g., single hardware platform, single branch), this role may be taken over by the same person as the team leader.
- A **tester** is responsible for ensuring that each subsystem works as specified by the designer. Often, development projects have a separate team responsible only for testing. Separating the roles of designer, implementor, and tester leads to more effective testing.

Consultant roles

Consultant roles address critical short-term needs of knowledge. Consultants roles are usually not involved full-time with the project. There are three types of consultant roles.

- The **client** is responsible for the formulation of scenarios and the requirements. This includes functional and nonfunctional requirements as well as constraints. The client is expected to be able to interact with the other developers' roles. The client represents the end user of the system.
- The **application domain specialist** is responsible for providing domain knowledge about a specific functional area of the system. Whereas the client has a global view of the functionality of the system, the application domain specialist has detailed knowledge of the day-to-day operation of a specific part of the system.
- The **technical consultant** is responsible for providing missing know-how to developers on a specific topic. This topic can be related to the development method, the process, implementation technology, or the development environment.

Promoters

When the organization is in place, roles and tasks have been assigned to participants, and participants have been trained, it becomes increasingly difficult to introduce changes in the project. To deal with this problem, especially in hierarchical organizations and functional organizations, promoter roles are essential in pushing an organizational change [Hauschildt & Gemuenden, 1998]. Promoters are self-appointed individuals who identify themselves with the outcome of the project. They are members of the corporate organization and may not necessarily be directly involved with the project. Instead, they are interfaces to the rest of the corporate organization. Because of their power, knowledge of technology, or familiarity with the project's processes, they are able to promote and push specific changes through the organization. Promoters seem like revolutionaries: Their main job is to embody change by getting rid of the old organization. However, this view is incorrect. Promoters, unlike revolutionaries, stay within the structure of the current organization. To overcome the barriers, they even use hierarchical power to move to the new process. There are three key promoters for any change to occur: the power promoter, the knowledge promoter, and the process promoter.

The **power promoter**, also called the executive champion, pushes the change through the existing organizational hierarchy. The power promoter convinces, encourages, creates enthusiasm, punishes, and rewards. The power promoter is not necessary at or near the top of the organization, but must have protection from top-level management; otherwise, project opponents might be able to prevent the success of the project. However, the power promoter does not avoid confrontation with top-level management. He or she needs to constantly identify difficulties, resolve issues, and communicate with the project members, especially with the developers.

The **knowledge promoter**, also called the technologist, promotes change arising in the application domain or the solution domain. He or she is a specialist because of specific capabilities and is usually associated with the power promoter. The knowledge promoter acquires information iteratively, understands the benefits and limitations of new technologies, and argues its adoption with the other developers.

The **process promoter** has intimate knowledge of the project's processes and procedures. The process promoter is in constant interaction with the power promoter to get consensus on the overall goals. In addition, the process promoter has sufficient managerial and technical knowledge to provide a bridge between the power and knowledge promoters, who often do not speak or understand the same language. An example of process promoter is the development lead, which is sometimes found in development teams. This role, not covered in detailed in this chapter, is responsible for the administrative aspects of a project, including planning, milestones definition, budgeting, and communication infrastructure.

11.3.3 Work products

Assigning a role to a participant usually translates into assigning the responsibility of a specific work product. A **work product** is an artifact that is produced during the development, such as an internal document for other project participants or a deliverable to the customer. An analyst is responsible for a section of the RAD describing the functionality of the system. A system architect is responsible for the SDD describing the system's high-level architecture. A project manager is responsible for the SPMP describing the development processes. Finally, the system and its accompanying documentation also constitute a set of work products that are delivered to the client. Complex work products, such as a RAD, can be treated as the composition of several smaller work products, such as the requirements for different types of users.

Work products are the result of tasks and are subject to deadlines. Work products are usually then fed into other tasks. For example, the test planning activity for the database subsystem results in a work product including a number of test suites and its expected results. The test suite is then fed to the testing activity of the given subsystem (Figure 11-7).

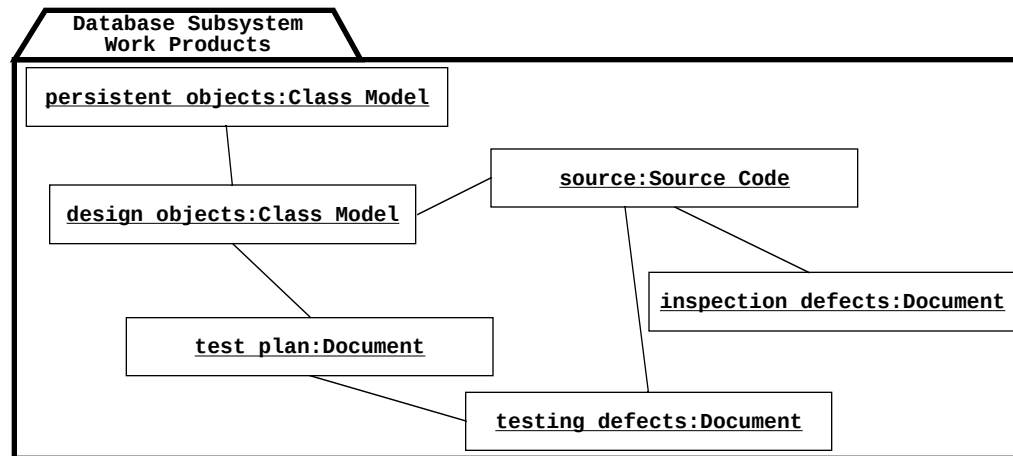


Figure 11-7 Work products for the a database subsystem team (UML object diagram). Associations represent dependencies among work products.

Work products are important management artifacts, because we can assess their delivery and the start of the tasks depending on specific work products. The late delivery of a testing suite for a subsystem, for example, will delay the start of its testing. Note, however, that focusing only on timely delivery is not sufficient: Rushing the delivery of test suites will meet the project schedule, but can also mean that critical faults are not discovered in time.

11.3.4 Tasks

A role describes work in terms of a set of tasks. A task includes a description, a duration, and is assigned to a role. A task represents an atomic unit of work that can be managed: A manager assigns it to a developer, the developer carries it out, and the manager monitors the progress and completion of the task. Tasks consume resources, result in a work products, and depend on work products produced by other tasks. Table 11-1 provides simple examples of tasks.

Table 11-1 Examples of tasks for the realization of the database subsystem

Task name	Assigned role	Task description	Task input	Task output
<u>Database subsystem requirements elicitation</u>	System designer	Elicits requirements from subsystem teams about their storage needs, including persistent objects, their attributes, and relationships	Team liaisons	Database subsystem API, persistent object analysis model (UML class diagram)
<u>Database subsystem design</u>	Subsystem design	Designs the database subsystem, including the possible selection of a commercial product	Subsystem API	Database subsystem design (UML diagram)
<u>Database subsystem implementation</u>	Implementor	Implements the database subsystem	Subsystem design	Database subsystem source code
<u>Database subsystem inspection</u>	Implementor	Conducts a code inspection of the database subsystem	Subsystem source code	List of discovered defects
<u>Database subsystem test plan</u>	Tester	Develops a test suite for the database subsystem	Subsystem design, subsystem source code	Tests and test plan
<u>Database subsystem test</u>	Tester	Executes the test suite for the database subsystem	Subsystem, test plan	Test results, list of discovered defects

Tasks are related by dependencies. For example, in Table 11-1, the Database subsystem test task cannot start before completion of the Database subsystem implementation task. Finding the dependencies between tasks enables us to find which tasks can be executed in parallel. For example, in Table 11-1, the Database subsystem inspection and the Database subsystem test plan can be assigned to two different developers and proceed at the same time. The set of tasks and their dependencies are called the task model. Figure 11-8 depicts in UML the task model corresponding to Table 11-1.

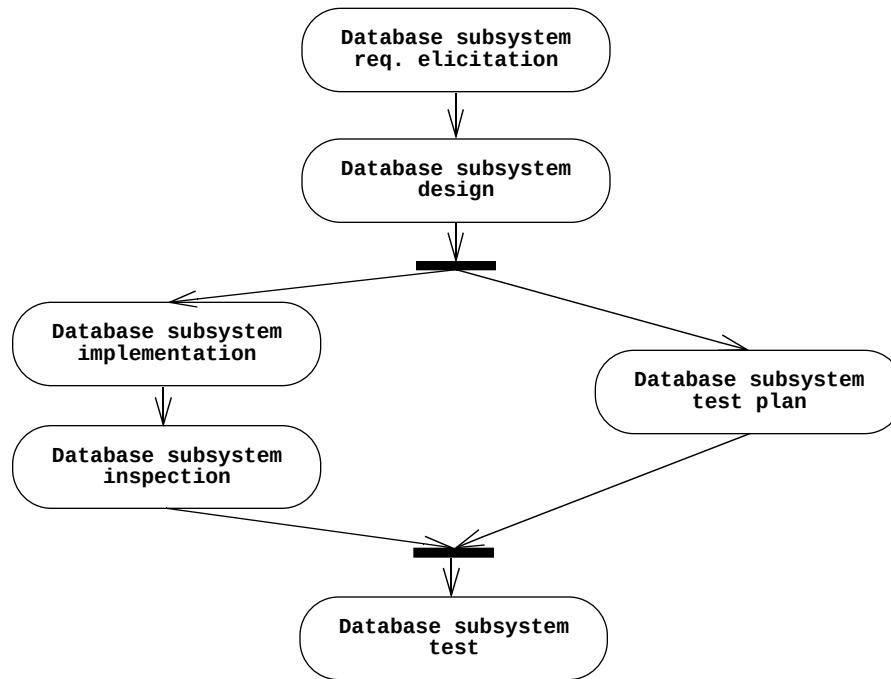


Figure 11-8 An example of a task model with precedence dependencies for the database subsystem example of Table 11-1 (UML activity diagram).

Task dependencies are relationships between individual tasks and time. Task constraints can be used to make sure a project is on schedule independent from the relationship of the task with other tasks. They usually represent an interaction with the client or the user. For example, if a task T has to start on a Monday, it has an MSO (see Table 11-2) constraint. If the system must be delivered in the first week of December, it has an MFO constraint. ASAP is the most commonly used constraint by people ordering tasks, (e.g., project managers), ALAP is the most commonly used constraint by people receiving tasks.

11.3.5 Schedule

The task model contains the tasks, their dependencies, their constraints, and their planned duration. Similar to object models, where objects and object dependencies are illustrated, there are many ways to represent task models. A schedule is the mapping of tasks onto time: Each task is assigned planned start and end times. This allows us to plan the deadlines for individual deliverables. The two most often used diagrammatic notations for schedules are PERT and Gantt charts [Hillier & Lieberman, 1967]. A **Gantt chart** is a compact way to present the schedule of a software project along the time axis. A Gantt chart is a bar graph, where the horizontal axis

Table 11-2 Task constraints

Constraint	Definition
ASAP	Start the task as soon as possible
ALAP	As late as possible
FNET	Finish no earlier than
SNET	Start no earlier than
FNLT	Finish no later than
SNLT	Start no later than
MFO	Must finish on
MSO	Must start on

represents time. The vertical axis represents the different tasks to be done. Tasks are represented as bars whose length correspond to the planned duration of the task. A schedule for the database subsystem example is represented as a Gantt chart in Figure 11-9.

A **PERT chart** represents a schedule as an acyclic graph of tasks. Figure 11-10 is a PERT chart for the database subsystem. The planned start and duration of the tasks are used to compute the critical path, which represents the shortest possible path through the graph. The length of the critical path corresponds to the shortest possible schedule, assuming sufficient resources to accomplish in parallel tasks that are independent. Moreover, tasks on the critical path are the most important, as a delay in any of these tasks will result in a delay in the overall project. The tasks and bars represented in thicker lines belong to the critical path.

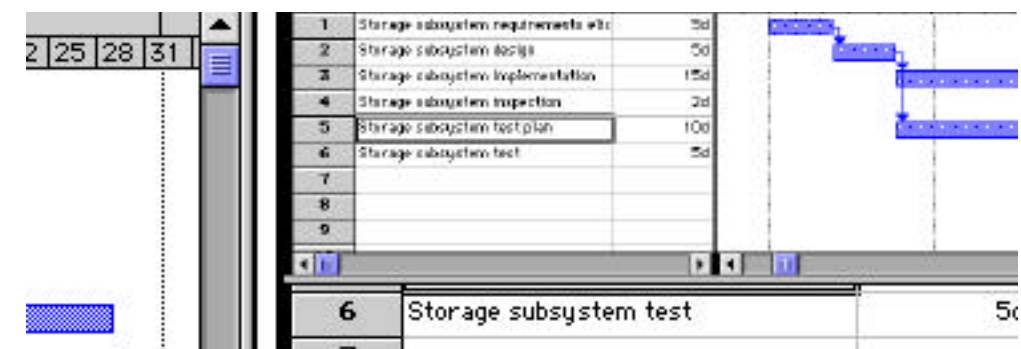
**Figure 11-9** An example of schedule for the database subsystem (Gantt chart).



Figure 11-10 An example of schedule for the database subsystem (PERT chart). Tasks on the critical path are represented with thick lines.

PERT charts and Gantt charts are useful tools for planning a project and tracking its execution. Note, however, that these models are only as good as the estimations they represent. Accurate estimation of the duration of a task is difficult. Project managers use their experience, data from past projects, and their intuition to estimate the tasks required for the execution of the project and their duration. Inaccurate estimation, such as the omission of a key task or the underestimation of a task on the critical path, leads to substantial delays in the schedule. Unexpected events, such as change in requirements or the late discovery of design errors, disrupt the schedule, no matter how accurate the planning. A project manager must try to anticipate these unwanted situations by allowing error margins and leaving room for change in the schedule. A task dedicated to the periodic review and change of requirements during the project is one such example. In general, PERT and Gantt charts are of limited usefulness if the tasks are not known during planning or if the tasks change frequently.

In the next section, we describe how project management organizes participants into teams, assigns them roles, plans the project in terms of tasks, and most important, revises the organization, role assignment, and tasks as the project progresses and encounters unanticipated events.

11.4 Project management activities

In this section, we describe the activities of project management. These include all the activities necessary to define and revise the organization, roles, work products, tasks, and schedule of the project. Most of the team leader effort is spent in monitoring and managing a team during steady state. Team leaders should be aware, however, of the project initiation activities preceding their involvement with the project and of the interaction with the client.

We focus on the following management activities:

- **Project initiation** (Section 11.4.2), including defining the problem, identifying the initial task plan, and allocating resources to tasks
- **Steady state**, including project monitoring (Section 11.4.3), risk management (Section 11.4.4), and the project agreement (Section 11.4.5)
- **Project termination**, including the client acceptance test (Section 11.4.6), installation (Section 11.4.7), and project postmortem (Section 11.4.8)

We illustrate the project management activities using the OWL project as an example [Bruegge et al., 1997]. The goal of OWL is to provide a system for intelligent buildings, allowing workers to adjust their environment, facilities managers to rapidly detect faulty components, and companies to reconfigure the floor plan based on their evolving needs. OWL entails the use and integration of new technology, including sensors for monitoring work's environment, actuators for controlling the temperature, humidity, and lighting of the work environment, distributed real-time control kernels, and Web technology. The requirements of OWL have not yet stabilized, as different manufacturers are still being considered. These factors will cause in several changes late in the development and introduce significant risk into the project, and thus OWL serves as the basis of the examples in this section.

11.4.1 Intelligent office buildings

An intelligent office building aims at providing individualized services to its workers while optimizing resource consumption. Workers in intelligent buildings are able to adjust many parameters of their individual environment, such as light level, temperature, and speed and direction of airflow, similar to what they already do in their cars. Moreover, a modular design enables workspaces to be rapidly reconfigured to meet changing organization or technological needs. Adding a computer or a video conference unit to a workspace can be done with limited rewiring. Finally, control systems integrated with intelligent buildings make their maintenance easier: Burned out light bulbs or defective components are detected remotely and replaced with minimum latency.

A critical component of intelligent buildings is their control system. The control system is a distributed real-time system that:

- responds to worker specified parameters by controlling shades, individual heating and cooling units, and windows based on the current climate to adjust temperature and airflow
- records building data for future analysis
- performs diagnostics
- notifies facilities managers about any problems or emergencies



Figure 11-11 Floor plan for the Intelligent Workplace, an intelligent building in the Department of Architecture in Carnegie Mellon University.

Such control systems also need to be modular and modifiable to accommodate new components and equipment from a variety of manufacturers. Figure 11-11 displays the floor plan for the Intelligent Workplace, an intelligent building at Carnegie Mellon University [Hartkopf et al., 1997]. In this chapter, we describe project management examples from OWL, a control system for the Intelligent Workplace.

11.4.2 Initiating the project

Project initiation focuses on defining the problem, planning a solution, and allocating resources. Project initiation results in the following work products (Figure 11-12):

- The **problem statement** is a short document describing the problem the system should address, the target environment, the client deliverables, and acceptance criteria. The problem statement is an initial description and is the seed for the *Project Agreement*, formalizing the common understanding of the project by the client and management, and for the RAD, a precise description of the system under development.
- The **top-level design** represents the initial decomposition of the system into subsystems. It is used to assign subsystems to individual teams. The top-level design is also the seed for the SDD, documenting the software architecture.
- The **organization** describes the initial teams of the project, their roles, and their communication paths. The **initial task plan** and the **initial schedule** are initial descriptions of how resources should be allocated. The organization, the initial task plan, and the initial schedule are the seeds for the SPMP, which documents all managerial aspects of the project.

During project initiation, management assembles the teams according to the top-level design and the selected organization, sets up the communication infrastructure, and then kicks off the project by organizing the first meeting. Once all participants are assigned tasks and status reporting mechanisms are in place, the project is considered to be in steady state.

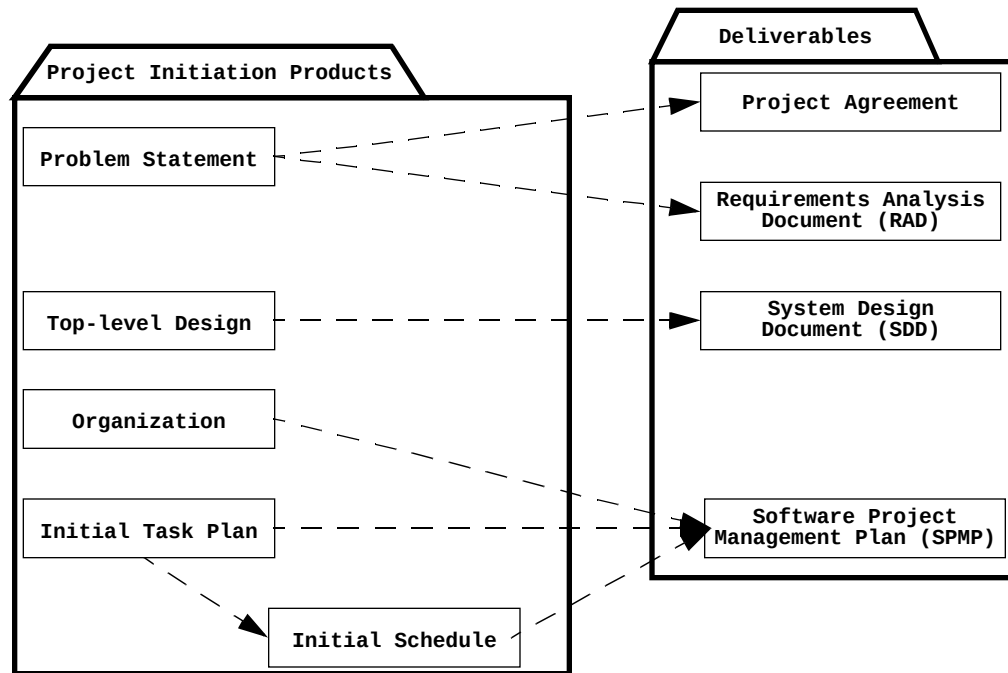


Figure 11-12 Work products generated during project initiation and their relationship with typical project deliverables (UML class diagram).

Developing the problem statement

The *Problem Statement* is developed by the project manager and the client as a mutual understanding of the problem to be addressed by the system. The problem statement describes the current situation, the functionality it should support, and environment in which the system will be deployed. It also defines the deliverables expected by the client, together with delivery dates and a set of acceptance criteria. The problem statement may also specify constraints on the development environment, such as the programming language to be used. The problem statement is not a precise or complete specification of the system. Instead, it is a high-level summary of two project documents yet to be developed: the RAD and the SPMP.

The problem statement is developed iteratively by the project manager and the client. Developing the problem statement is both a negotiation activity and an information gathering

activity during which each party learns the expectations and constraints of the other. Whereas the project statement contains a high-level description of the functionality of the system, it should also provide concrete examples to ensure that both parties are sharing the same vision. This is best done by using scenarios (see Chapter 4, *Requirements Elicitation*) for describing the current situation and future functionality.

Figure 11-13 is an example outline for a problem statement. The first section describes the problem domain. Section 2 of the problem statement provides example scenarios describing the interaction between users and the system for the essential functionality. Scenarios are used both for describing the current situation and the future situation.

PROBLEM STATEMENT

1. Problem domain
 2. Scenarios
 3. Functional requirements
 4. Nonfunctional requirements
 5. Target environment
 6. Deliverables & deadlines
-

Figure 11-13 Outline of problem statement document. Note that the sections 2, 3, and 4 are not a complete or precise specification of the system. They provide the basis for requirements elicitation. Section 6, however, describes contractual deliverables and their associated due dates.

Figure 11-14 provides examples of scenarios used in the OWL problem statement. Section 3 is a summary of the functional requirements of the system. Section 4 is a summary of the nonfunctional requirements, including client constraints such as programming language and component reuse. Section 5 is a description of the deployment environment, including platform, physical environment, users, and so on. Section 6 is a list of client deliverables and their associated deadlines.

Defining the top-level design

The top-level design describes the software architecture of the system. In a project- or team-based organization, the top-level design is used as basis of the organization: Each subsystem is assigned to a team that is responsible for its definition and realization. The subsystem decomposition is refined and modified later during system design (see Chapter 6, *System Design*). We only need, however, to identify the major subsystems and their services; we need not define their interfaces at this point. Teams who work on dependent subsystems will negotiate individual services and their interfaces as they are needed.

The top-level design of OWL includes five major subsystems, described in Figure 11-15. Note that this subsystem decomposition is high level and focuses on functionality. Other subsystems may be added later in the project, such as a communication layer for moving objects among nodes in the network and a notification subsystem for pushing notices to users. Once

OWL PROBLEM STATEMENT**1. Problem domain**

A current trend in the building industry is to provide distributed services and control for the individual occupant as a strategy to correct the overreliance on large centralized systems that characterize office buildings built in the last 30 years. At the Intelligent Workplace workers will have more control over their environmental conditions—adjusting light level and temperature of their workspace, reducing glare, controlling speed and direction of air flow delivered to workspace. (You can do that in your car—why not in your office?) An energy-efficient facade will allow for fresh air ventilation from operable windows and incorporate movable shading devices that adjust to minimize glare and maximize natural lighting of the workspace.

It is desirable to adopt three forms of control in the Intelligent Workplace: responsive, scheduled, and user driven. Responsive control is when the system reacts to a change in sensor reading by actuating some components. Scheduled control can be adopted in the presence of predictable data that allows the components to be directly controlled by a carefully designed schedule. For example, because the position of the Sun is predictable, a schedule for the interior shades of the Intelligent Workplace can be adopted. Control system should be flexible enough to respond to the needs of the occupants. If they would like to change the temperature of their local environment, they should be given that opportunity.

In this project, you are asked to build a system called OWL (Object-Oriented Workplace Laboratory) that attempts to improve the way we deal with buildings.

[. . .]

2. Scenarios**2.1 Building control**

The building occupant uses a Web browser to access his Personal Environment Module (PEM). He adjusts the temperature and airspeed to cool his workspace. The control information is sent to the PEM equipment. The control actions are logged in the database and the equipment adjusts the heater and the ventilation of the workspace. The system checks neighboring PEMs to check if cooling this particular workspace requires other workspaces heating to be increased.

[. . .]

2.5 Building maintenance

The system monitors the behavior of the controlled devices to detect faults in the system. Faulty light bulbs and unusual parameter readings are reported to the facilities manager who then plans inspections and repairs. The occurrences of device faults are logged and analyzed for trends, enabling the facilities manager to anticipate faults in the future.

[. . .]

Figure 11-14 Excerpts from the problem statement of OWL [OWL, 1996].

analysis produces a stable model and the subsystem decomposition is revisited during subsystem design, new subsystems can be created during the system design phase, and the organization might have to be changed to reflect the new design.

Identifying the initial tasks

The starting points for task identification are the problem statement (including deliverables and deadlines), the life cycle activities, and past experience. The client and the

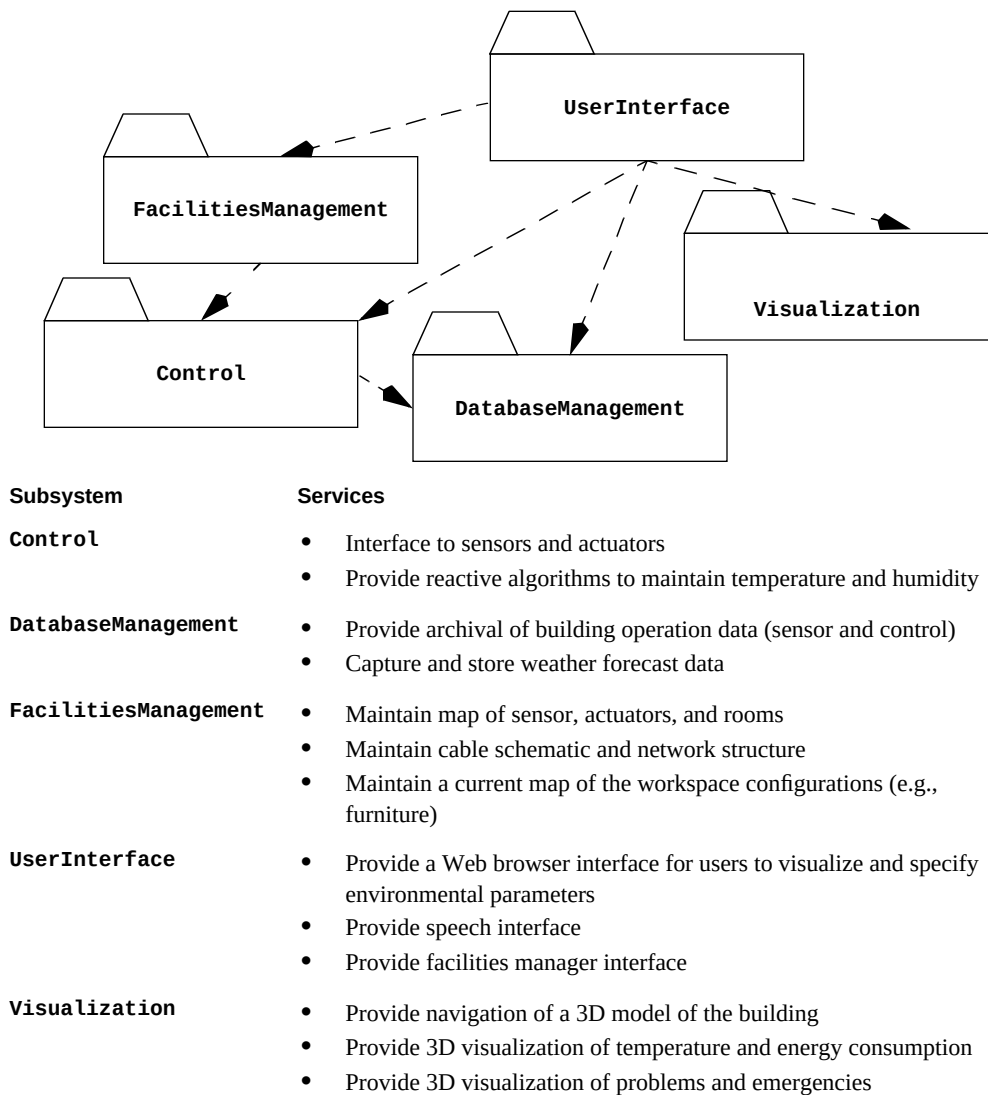


Figure 11-15 Top-level design of OWL (UML class diagram, packages collapsed).

project manager agree to a series of reviews whose purpose is to review the progress of the project. Each deliverable must be covered by least one task.

The planning activity identifies additional work products for the project's own consumption. These include management work products, such as the SPMP, development work

products, such as preliminary interface definitions, rapid prototypes, and evaluations, such as summary sheets for tools, methods, or procedures.

The work is initially decomposed into tasks that are small enough to be estimated and assigned to a single person. This initial decomposition is often called the work breakdown structure (WBS) and resembles a large To Do list. Usually, the more specific the requirements and the deliverables, the easier it is to define precise tasks. When requirements are not well defined, the manager must allocate additional time for replanning and revising the task plan as the requirements stabilize.

For example, let us consider the design activities for the team who develops the DatabaseManagementsubsystem of OWL. From the requirements, the team knows that the DatabaseManagementsubsystem should reliably maintain the state of sensors and the actions issued to the actors. Moreover, the DatabaseManagementsubsystem should support searches over historical data for trend analysis. Table 11-3 depicts an example of an initial work breakdown structure for such a subsystem. Estimated times are based on prior experience of the manager or the estimator.

The initial work breakdown structure should not be too detailed. Many project managers have a tendency to create detailed To Do lists at the beginning of the project. Even though a detailed model of work effort is desirable, it is difficult to create a meaningful detailed work breakdown structure initially. Refining the system definition during analysis and designing the software architecture during system design generate many changes to the original work breakdown structure, including new tasks to investigate recent technology, new subsystems, and additional functionality. Creating a detailed work breakdown structure only leads to additional work when the plan is revised. Instead, a manager details the work breakdown structure only for the short term while describing the longer term tasks in less detail. The manager revises and details the work breakdown structure only as the project progresses.

Identifying task dependencies

Once tasks have been identified, we need to find the dependencies among them. Identifying dependencies allows us to assign resources to tasks more efficiently. For example, two independent tasks can be assigned to different participants to be accomplished in parallel. We find dependencies among tasks by examining the work products each task requires. For example, a unit testing task requires the specification of the unit programmer interface, generated by the system design task. The system design task must then be completed before the testing task can be initiated. Figure 11-16 depicts the dependencies among the tasks identified in Table 11-3. The set of tasks and their dependencies constitute the task model. The next step is to map the task model to resources and time by creating a schedule.

Creating the initial schedule

The initial schedule is created by mapping the task model to time and resources. Task dependencies, estimated task durations, and the number of participants are used for creating the initial schedule. The initial schedule is used to plan dates for the primary interactions with the

Table 11-3 An example of initial work breakdown structure for DatabaseManagement

Task	Estimated time
1. Select database management system	2 weeks
2. Identify persistent objects and their attributes	1 week
3. Identify queries	1 week
4. Identify searchable attributes	1 day
5. Define schema	2 weeks
6. Build prototype for performance evaluation	2 weeks
7. Define API to other subsystems	3 days
8. Identify concurrency requirements	2 days
9. Implement database subsystem	2 weeks
10. Unit test database subsystem	3 weeks
11. Address remaining concurrency hazards	2 weeks

client and the users, including user interviews, client reviews, and deliveries. These dates are made part of the problem statement and represent deadlines mutually agreed on by the client and the project manager. These dates are planned such that, even in the event of change, the deadlines can still be met. Figure 11-17 depicts the initial schedule for OWL.

Similar to the task model, it is unrealistic to create a detailed schedule at the beginning of the project. The initial schedule should include deadlines for all client deliverables and detailed scheduling for the short term (e.g., the first two months). Detailed scheduling for the rest of the project is performed only as the project progresses. Also, once the project is underway, each team can contribute to the planning of its tasks, given that it has a more detailed view of the work to be accomplished. The overall scheduling is done as part of a continuing trade-off between resources, time, and implemented functionality and is updated constantly.

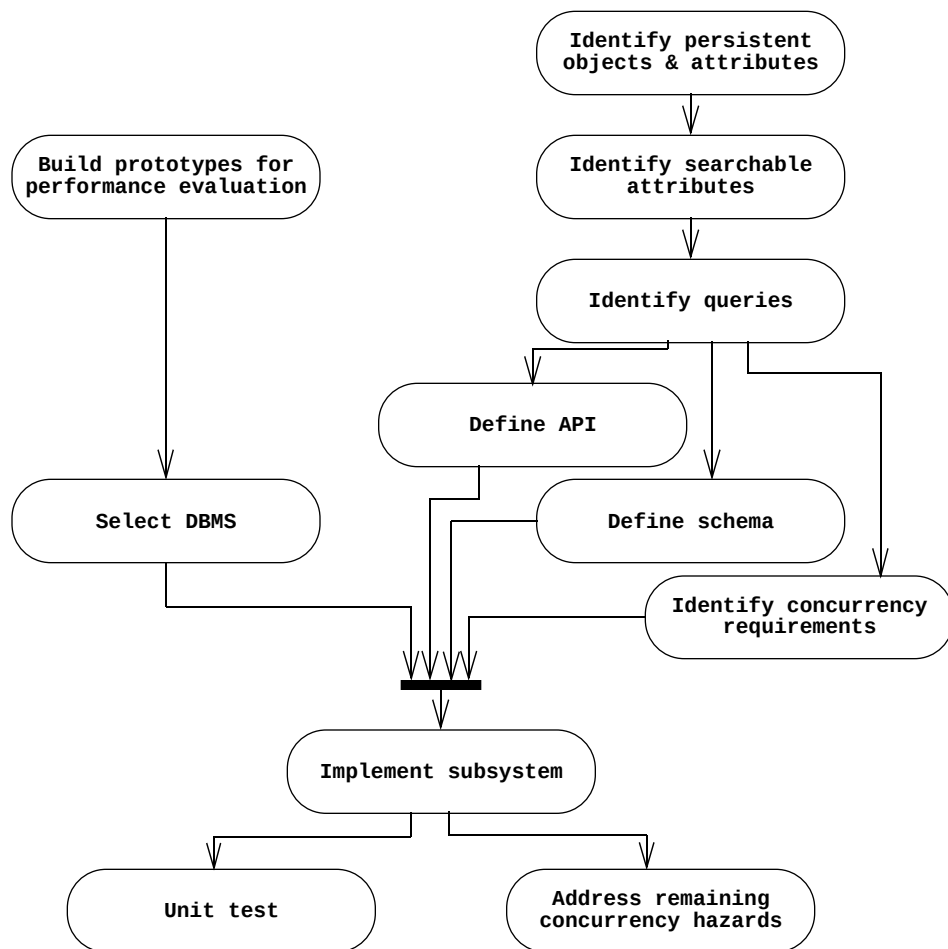


Figure 11-16 Task model for the work breakdown structure of Table 11-3.

Assembling the teams

The next step of project initiation is to assemble the teams who will produce the deliverables. Developers who will work for the project can be assigned to the project all at once (flat staffing), or the project can be gradually ramped up by hiring people as needed (gradual staffing). On the one hand, gradual staffing is motivated by saving resources in the early part of the project. Requirements elicitation and analysis do not require as many people as coding and testing do. Moreover, analyst and implementor are roles requiring different skills and thus should be assigned to different people. On the other hand, flat staffing has the advantage of establishing teams early and the social environment necessary for spontaneous communication.

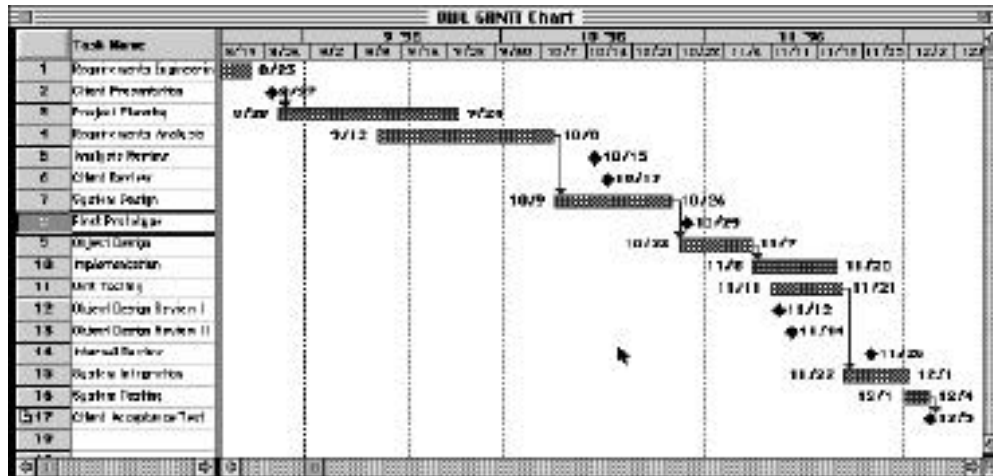


Figure 11-17 Initial project schedule for OWL (Gantt chart). The initial schedule serves to estimate the delivery dates and the interaction with the client. The schedule is detailed and revised as the project progresses.

Some of the developers can be assigned to the analysis activities with the analysts, while the others may already start other activities such as setting up the configuration management environment, technology survey and evaluation, and training. With shorter projects and market deadlines, flat staffing is becoming the preferred staffing scheme. The dilemma between gradual staffing and flat staffing has been discussed in length by Brooks [Brooks, 1975].

The project manager staffs teams by examining the top-level design and considering the skills required for each subsystem. Often, not enough developers with the required skills are available, in which case the project manager includes time in the task model for the technical training of new staff. The project manager distributes new staff across the project such that each team has at least one experienced developer. The experienced developer provides technical leadership and serves as a mentor for the new developers who then gain experience more quickly when putting their new training into practice. Formal training and learning by practice is a trade-off the project manager evaluates given project time constraints. Formal training enables developers to become more productive earlier; however, it is expensive and constitutes a longer term investment benefiting future projects.

The project manager selects the team leaders before the teams are formed. In addition to being capable of understanding the team status, the team leader needs to be able to communicate effectively, recognize pending crises (social or technical), and make trade-offs taking into account project level concerns. The start of the project is also an ideal time for training team leaders, for familiarizing them with project procedures and ground rules. The team leader role is distinct from the role of technical leader. The technical lead, usually the liaison to the

architecture team, interacts with the chief architect and the architecture liaisons from the other teams and has final say on the technical decisions within the team. The role of architecture liaison requires excellent technical skill and development experience.

In a training context, the team leader can be split into two roles. A coach trains, explains, and advises the team. The coach also acts as the liaison to management. A team leader trainee fills all other responsibilities of the team leader, including facilitating status meetings and assigning tasks. In a course project, this role is filled by the instructor or the teaching assistant, whereas the architecture liaison is filled by a student. In a development project, the team leader is filled by a developer who has experience and knowledge with the organization's management procedures, whereas the architecture liaison is filled by a developer who has strong technical skills.

Table 11-4 depicts an example of role assignment to participants for the database team of the OWL project. The project manager and team leaders assign roles based on skills and determine the training needs for each participant.

Table 11-4 Role assignment, skills, and training needs for the database team of OWL

Participant	Roles	Skills	Training needs
Alice	Team leader	Management: team leader Programming: C Configuration management	UML Communication skills
John	Architecture liaison Implementor	Programming: C++ Modeling: UML	Java
Mary	Configuration manager Implementor	Programming: C++, Java Modeling: Entity relationship Databases: relational Configuration management	Object-oriented databases UML modeling
Chris	Implementor	Programming: C++, Java Modeling: Entity relationship Databases: object-oriented	UML modeling
Sam	Facilities management liaison Tester	Programming: C++ Testing: white box, black box	Inspections Java

Setting up the communication infrastructure

Clear and accurate communication is critical for the success of a development project. Moreover, communication becomes crucial when the number of project participants increases. Consequently, setting up the communication infrastructure for a project occurs as early as possible, that is, during project initiation. The project manager needs to address the following modes of communication, as described in Chapter 3, *Project Communication*:

- *Scheduled modes of communication.* These include planned milestones, such as client and project reviews, team status meetings, inspections, and so on. Scheduled modes of communication are the formal means through which project participants gather and share information. These are best supported by synchronous or face-to-face communication, such as meetings, formal presentations, video conferences, and telephone conference calls. The manager schedules dates and designates facilitators for each of these interactions.
- *Event-based modes of communication.* These include problem reports, change requests, issue discussion, and resolution. Event-based modes usually arise from unforeseen problems and crises. Asynchronous mechanisms such as E-mail, groupware, and problem databases need to be set up early, and participants need to be trained to use them. When the number of participants is large, centralized infrastructures, such as Web sites and bulletin boards are preferable as they make more information visible to more people than E-mail or bilateral conversations do. For example, Figure 11-18 is the home page of the OWL project, providing access to the project address book, bboards, documents, and source code. This infrastructure also has the advantage of being accessible with minimum software and platform requirements.

The project manager uses different mechanisms of communication with the client and the end users than with the developers. Developers, on the one hand, and clients and end users, on the other hand, have different terminology, information needs, and availability. The client and end users speak the application domain language and may not have much software engineering background. They need information about the status of the project and of the analysis model. Their involvement with the project is thus episodic and focuses on their own work. Developers speak the language of the solution domain and may not have much application domain knowledge. Their information need is technical and daily. A good heuristic is to conduct communication face to face several times during the project and to establish a client bboard for unscheduled communication needs (e.g., for clarification of requirements). This assumes that the client is familiar with the use of bboards and on-line communication. Training or other mechanisms or communication are used otherwise. Note, however, that the communication channel between the client and the users has a low bandwidth and needs to be used efficiently.

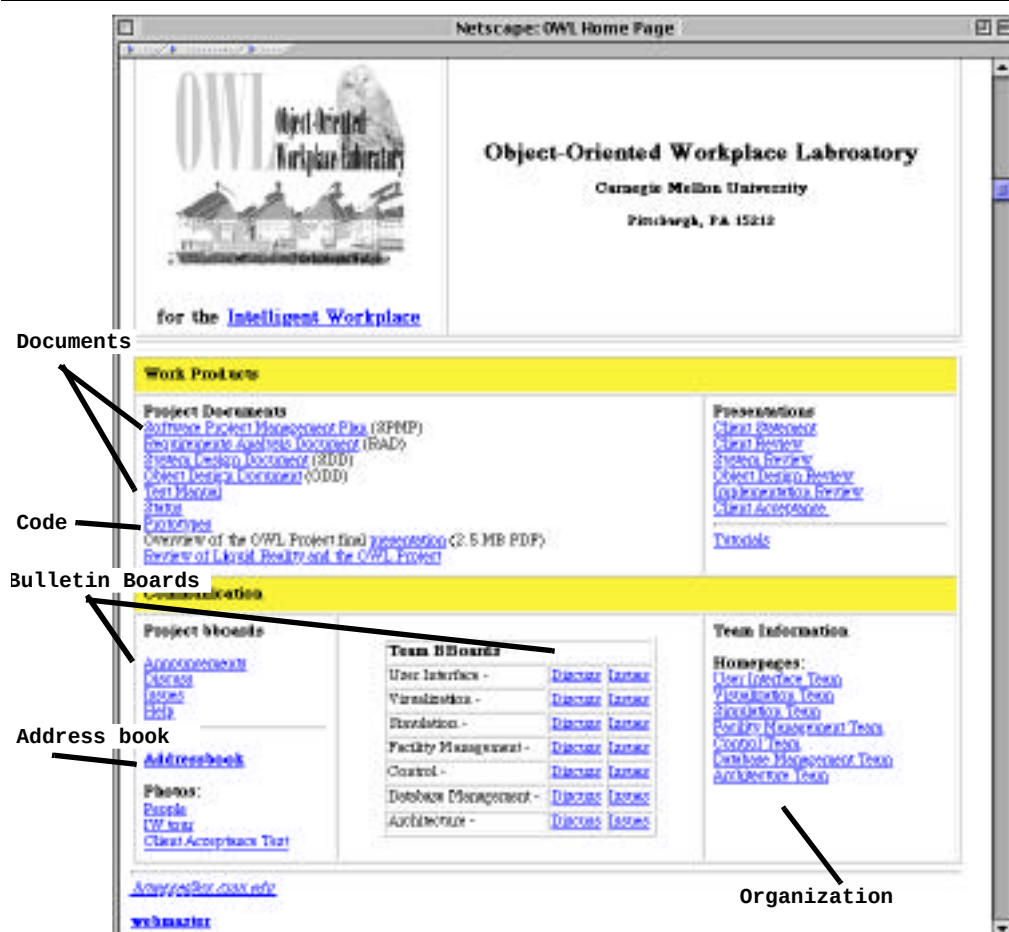


Figure 11-18 Home page for OWL. Developers access all relevant information from a central Web page.

Kicking off the project

The project kickoff meeting marks the end of the project initiation phase and the beginning of the development phase. The project manager has completed much of the initial planning and top-level design, selected an organization, hired developers, and designated team leaders. The project kickoff consists of three meetings.

- *The client presentation.* All project participants are included in this meeting. It includes a presentation by the client on the system requirements and a presentation by the manager on the initial organization and schedule.
- *The management kickoff meeting.* This meeting is facilitated by the project manager and including all the team leaders. This meeting serves to define and train managerial

procedures, such as meeting procedures, configuration management, and problem reporting.

- *Individual team kickoff meetings.* These meetings are facilitated by team leaders and includes the team members. Procedures defined during the management meeting are communicated to the participants. Team members are introduced to each other. This is where the involvement of a team leader begins. Figure 11-19 is an example of an agenda for the database team kickoff meeting.

The goal of the three kickoff meetings is for participants to get to know each other and to initiate communication. Participants become familiar with basic meeting procedures, the organization and the assignment of roles, and the general mechanisms of the project. Soon after, tasks are distributed and regular status meetings take place. After a small number of status meetings, the scheduled aspects of communication should be solidly in place.

Next, we describe the activities of project monitoring during steady state. The management activities during steady state focus mostly on unexpected events and contingency plans.

11.4.3 Project monitoring

To make effective decisions, the project manager needs accurate status information. Unfortunately, collecting accurate status is difficult. While a complex system is difficult to understand in itself, the status of its components and its impact on future deliveries are even more difficult to understand. Developers will not report to their team leader any problem they believe they can address in time. However, small departures from the schedule, not worth reporting independently, aggregate and degenerate into large departures much later in the schedule. By the time team leaders discover and report a major problem to the project manager, it has already caused a substantial delay in the project. Several tools are available to collect status information. Because none of them are accurate or reliable alone, team leaders and project managers need to use a combination of these. Below we review their advantages and disadvantages.

Meetings

- *Periodic status meetings.* Status meetings conducted at the team level have the best potential for reporting status and information required for corrective decisions. Status meetings, however, can also provide inaccurate status information if team members do not cooperate. Participants are naturally reluctant to report problems or mistakes. They cooperate only if they can trust a manager not to step into problems that they can resolve themselves. Managers need to make problem reporting beneficial to developers by intervening only when necessary.
- *Sharp milestones.* Progress can be measured by determining if developers deliver work products on time. Managers can increase the accuracy of this method by defining sharp milestones such that they can be accurately monitored. For example, the “coding completed” milestone is not a sharp milestone, because it does not take into account the

AGENDA: Database team kickoff meeting

When and Where	Role
Date: 11/13/1998	Primary Facilitator: Mary
Start: 4:30pm	Timekeeper: John
End: 5:30pm	Minute Taker: Chris
Building: AC Hall	Room: 3421

1. Purpose

Become familiar with project management roles for a medium-scale project with a 2-level hierarchy.

2. Desired outcome

- Group roles are assigned to people
- Meeting times are finalized
- First set of action items for next meeting

3. Information sharing [Allocated time: 25 minutes]

Meeting procedures

- guidelines
- agendas/minutes
- roles

Team roles

- Team leader
- Architecture Liaison
- Documentation Liaison
- Configuration Manager
- Toolsmith

Team tasks

Team schedule

Problem reporting

4. Discussion [Allocated time: 25 minutes]

Define ground rules.

Initial role and task assignment

5. Wrap up [Allocated time: 5 minutes]

Review and assign new action items.

Meeting critique.

Figure 11-19 An example of an agenda for the database team kickoff meeting in the OWL project.

quality of the code delivered. The code could contain few or many defects and the milestone would be considered completed in either case. Conversely, defining a milestone as the completion of the test, demonstration, documentation, and integration of a specific feature yields better status information. When defining and monitoring sharp milestones, managers do not need the cooperation of developers.

- *Project reviews.* A project review is a means for all participants to exchange status and plan information about all the teams with formal presentation. Project reviews suffer from similar problems as status meetings: If participants are already reluctant to admit problems in front of their team leader, they definitively will not admit problems in a public forum.
- *Code inspections.* Formal peer reviews of code are an effective methods for discovering defects early. When performed frequently, their results can also be used as an indication of progress.
- *Prototype demonstrations.* Prototype demonstrations are partial implementations of the system for evaluating technology or functionality. They usually do not illustrate how border cases are dealt with or how robust the implementation is. They are good for measuring initial progress but are not sharp measures of completion.

Metrics

- *Defects to be resolved.* Once an initial version of the system exists, the number of defects to be resolved is an intuitive measure of how much effort needs to be expended before the system can be delivered. However, measuring the number of open defects encourages their quick resolution and triggers the introduction of new defects, thus defeating the original purpose of measuring the number of defects.
- *Source code metrics.* Various metrics on source code, ranging from the number of lines of code to ratios of the number of operators and operands and to function points, have been proposed for estimation. These metrics are sensitive to many factors in the environment, including the coding style of the implementers. Moreover, results do not seem transferable from one organization to the other. These measures should only be used if a systematic effort, across multiple projects, is organized within the organization.

Finally, independently of the methods used for determining status, the project manager, the team leaders, and the developers need to communicate status information in understandable terms. Next, we describe risk management as a method for controlling projects, communicating potential problems, and planning contingencies.

11.4.4 Risk management

The focus of risk management is to identify possible problems in the project and address them before they can impact significantly the delivery date or the budget. The key element of risk management is to set up the appropriate information flows such that risks and problems are accurately reported in a timely manner. Many projects fail either because simple problems were reported too late or because the wrong problem was addressed. In this section, we focus on the activities of risk management for identifying, analyzing, and addressing risks. For more details on risk management in software engineering, we refer the reader to the specialized literature; for example, [Boehm, 1991] and [Charette, 1989].

Identifying risks

The first step of risk management is to identify risks. Risks are potential problems that result from an area of uncertainty. Risks can be managerial or technical. Managerial risks include any uncertainty related to the organization, work products, roles, or task plan. Technical risks include any uncertainty related to the system models, including changes of the functionality of the system, the nonfunctional requirements, the architecture, or the implementation of the system. In particular, technical risks include defects discovered late in the development. Table 11-5 depicts examples of risks in the OWL project.

Table 11-5 Examples of risks identified in the OWL project

Risk	Risk type
Manufacturer's Personal Environment Module (PEM) does not comply with published standard	Technical
Manufacturer's PEM delivery date is not later than planned	Managerial
Users are not willing to use Web browser to adjust temperature settings	Technical
Selected middleware too slow to meet performance requirement for data logging	Technical
Development of abstract PEM layer takes more time than scheduled	Managerial

Developers are often aware of the risks applicable to their tasks. The challenge is to encourage developers to report risks such that they can surface and be managed. This entails rewarding developers for reporting risks and problems and making the risk management activities directly beneficial to developers. Spontaneous risk reporting is usually not sufficient: Project participants, including the client, the developers, and the managers, are usually reluctant to communicate potential failures or shortcomings and optimistic about their outcome. A more-systematic approach to risk identification is to interview the project participants using a structured questionnaire: Participants are asked in group sessions to list the risks they anticipate with respect to specific tasks. Figure 11-20 depicts example questions from the Taxonomy-based Risk Identification process of the SEI [Carr et al., 1993]. In this example, the interviewer attempts to determine if there are any risks related to performance nonfunctional requirements. Depending on the answer to the first question, the interviewer can ask follow-up questions to make sure no other related risks are present. The rationale behind this questionnaire is to cover all the areas of development in which risks are typically found.

1. Requirements

...

d. Performance

...

[23] Has a performance analysis been done?

(Yes) (23.a) What is your level of confidence in the performance analysis?

(Yes) (23.b) Do you have a model to track performance through design and implementation?

Figure 11-20 Questions for eliciting performance risks in the Taxonomy-based Risk Identification process. If the question to question 23 is positive, questions 23.a and 23.b are asked of the developers [Carr et al., 1993].

Prioritizing risks

Systematic risk identification produces a large number of managerial and technical risks, some of them critical, others unimportant. Prioritizing risks allows managers to focus on the critical risks only. Risks are characterized by the likelihood they can become problems and by their potential impact to the project when they become problems. By using these two attributes, risks can be assigned to one of four categories:

- likely, high potential impact
- unlikely, high potential impact
- likely, low potential impact
- unlikely, low potential impact

The risks of the first category are those managers should worry about (likely, high potential impact). For these risks, developers and managers should draw contingency plans and monitor the risk carefully. If the likelihood of the risk increases, managers can activate the contingency plan and address the problem in time. Moreover, managers should monitor the risks of the second category (unlikely, high potential impact). Contingency plans do not need to be drawn for them unless their likelihood increases. Finally, the risks in the third and fourth

categories can be ignored unless sufficient resources are available for monitoring them. Table 11-6 is the ordering of the risks presented in Table 11-5.

Table 11-6 Prioritized risks for the OWL project

Risk	Likelihood	Potential impact
Manufacturer's Personal Environment Module (PEM) does not comply with published standard.	Not likely	High
Manufacturer's PEM delivery date is later than planned.	Not likely	High
Users are not willing to use Web browser to adjust temperature settings.	Likely for users spending less than 2 hours per day in front of a computer	High
Selected middleware too slow to meet performance requirement for data logging.	Not likely, low sampling frequency	High
Development of abstract PEM layer takes more time than scheduled.	Likely, first occurrence	High

Mitigating risks

Once risks have been identified and prioritized, mitigation strategies need to be designed for the critical risks. Mitigation strategies can include lowering the likelihood of the risk or decreasing its potential impact. Risks are usually caused by an area of uncertainty, such as missing information or lack of confidence in some information. Developers decrease the likelihood of a risk by further investigating the causes of the risk, by changing suppliers or components, or by selecting a redundant supplier or component. Similarly, developers reduce the impact of the risk on the project by developing an alternate or redundant solution. In most cases, however, mitigating a risk incurs additional resources and cost. Table 11-7 describes mitigation strategies for the risks presented in Table 11-5.

Table 11-7 Mitigation strategies for the OWL risks

Risk	Mitigation strategy
Manufacturer's Personal Environment Module (PEM) does not comply with published standard.	• Run benchmarks to identify nonconformance. • Investigate if nonconforming functions can be avoided.
Manufacturer's PEM delivery date is later than planned.	• Monitor risk by asking manufacturer for intermediate status reports.
Users are not willing to use Web browser to adjust temperature settings.	• Perform usability studies using mock-ups. • Develop alternate interface.
Selected middleware too slow to meet performance requirement for data logging.	• Monitor risk. Plan for a performance evaluation prototype.
Development of abstract PEM layer takes more time than scheduled.	• Increase the priority of this task with respect to other implementation tasks. • Assign key developers to this task.

Communicating risks

Once risks are identified, prioritized, and mitigated, the risk management plan needs to be communicated to all concerned. Risk management relies on timely communication. Spontaneous and timely reporting of potential problems needs to be encouraged. Risk management plans and technical documents are communicated using the same channels. Developers and other project participants review risks at the same time they review the technical content of the project. As mentioned earlier, communication is the most significant barrier when dealing with uncertainty.

11.4.5 Project agreement

The *Project Agreement* is a document that formally defines the scope, duration, cost, and deliverables for the project. The form of a *Project Agreement* can be a contract, a statement of work, a business plan, or a project charter. The *Project Agreement* is typically finalized sometime after the analysis model stabilized and the planning of the rest of the project is underway.

A *Project Agreement* should contain at least the following:

- list of deliverable documents
- criteria for demonstrations of functional requirements

- criteria for demonstrations of nonfunctional requirements, including accuracy, reliability, response time, and security
- criteria for acceptance

The *Project Agreement* represents the baseline of the client acceptance test. Any change in the functionality to be delivered, the nonfunctional requirements, the deadlines, or the budget of the project requires the renegotiation of the *Project Agreement*.

11.4.6 Client acceptance test

The purpose of the client acceptance test is the presentation of the system and the approval of the client according to acceptance criteria set forth in the *Project Agreement*. The result of the client acceptance test is the formal acceptance (or rejection) of the system by the client. Installation of the system and field tests by the client may already have occurred prior to this test. The client acceptance test constitute the visible end of the project.

The client acceptance test is conducted as a series of presentations of functionality and novel features of the system. Important scenarios from the *Problem Statement* are exercised and demonstrated by the developers or by future users. Additional demonstrations focus on the nonfunctional requirements of the system, such as accuracy, reliability, response time, or security. If installation and user evaluations occurred prior to the client acceptance test, their results are presented and summarized. Finally, the client acceptance test also serves as a discussion forum for subsequent activities such as maintenance, knowledge transfer, or system enhancement.

11.4.7 Installation

The installation phase of a project includes the field test of the system, the comparison of the system results with the legacy system, the rollout of the legacy system, and the training of users. The installation may be conducted by the supplier or by the client, depending on the *Project Agreement*.

To minimize risks, installation and field testing are usually done incrementally, with noncritical sites used as a field test environment. It is only when the client is convinced that the disruption to his business will be kept to a minimum that the delivered system enters full-scale operation. Replacement systems and upgrades are rarely introduced in a “big-bang” fashion, as the largest number of problems are discovered during the first few days of operation.

11.4.8 Postmortem

Every project uncovers new problems, unforeseen events, and unexpected failures. Each project, therefore, constitutes an opportunity for learning and anticipating new risks. Many software companies conduct a postmortem study of each project after its completion. The postmortem includes collecting data about the initial planned delivery dates and actual delivery dates, number of defects discovered, qualitative information about technical and managerial

problems encountered, and suggestions for future projects. Although this phase is the least visible in the life cycle, the company depends on it the most for learning and improving its efficiency.

11.5 Managing project management models and activities

Project management activities, as technical activities, need to be documented, assigned, communicated, and revisited when unexpected events arise. In this section, we describe the management issues that apply to project management activities, including

- documenting management models (Section 11.5.1)
- assigning responsibilities for controlling and monitoring the project (Section 11.5.2)
- communicating about project management (Section 11.5.3)

11.5.1 Documenting project management

The management models described in Section 11.3 are documented in the SPMP [IEEE Std. 1058.1-1993]. The audience of the SPMP includes the management and the developers. The SPMP documents all issues related to client requirements (such as deliverables and acceptance criteria), the project goals, the project organization, the division of labor into tasks, and the allocation resources and responsibilities. Figure 11-21 depicts an example of outline for the SPMP.

The first section of the SPMP, *Introduction*, provides background information for the rest for the document. It briefly describes the project, the client deliverables, the project milestones, and the changes that the document can be expected to go through. This section contains the hard constraints found the *Project Agreement* that are relevant to the developers.

The second section of the SPMP describes the *Project organization*. The top-level design of the system is described, along with the subsystem and cross-functional teams that make up the project. The boundaries of each team and of the management are defined and responsibilities assigned. Communication roles, such as liaisons, are described in this section. By reading this section, developers are able to identify the participants in other teams they need to communicate with.

The third section of the SPMP, *Managerial process*, describes how management monitors project status and how it addresses unforeseen problems. Dependencies among teams and subsystems are described, anticipated risks and contingency plans are made public. Documenting what could go wrong makes it easier for the developers to report it when problems do occur. This section should be regularly updated to include newly identified risks.

The fourth section of the SPMP, *Technical process*, describes the technical standards that all teams are required to adopt. These range from the development methodology, to the configuration management policy of documents and code, to coding guidelines, and the selection of standard off-the-shelf components. Some might be imposed by companywide interests and others by the client.

Software Project Management Plan (SPMP)

1. Introduction
 - 1.1 Project overview
 - 1.2 Project deliverables
 - 1.3 Evolution of this document
 - 1.4 References
 - 1.5 Definitions and acronyms
 2. Project organization
 - 2.1 Process model
 - 2.2 Organizational structure
 - 2.3 Organizational boundaries and interfaces
 - 2.4 Project responsibilities
 3. Managerial process
 - 3.1 Management objectives and priorities
 - 3.2 Assumptions, dependencies and constraints
 - 3.3 Risk management
 - 3.4 Monitoring and controlling mechanisms
 4. Technical process
 - 4.1 Methods, tools, and techniques
 - 4.2 Software documentation
 - 4.3 Project support functions
 5. Work elements, schedule, and budget
-

Figure 11-21 An example of a template for the SPMP

The fifth section of the SPMP, *Work elements, schedule, and budget*, represents the most visible product of management. This section details how work is to be carried out and who should carry it out. The first versions of the SPMP contains detailed plans only for the early phases of the project. Detailed plans for later phases are updated as the project progresses and important risks are better understood.

The SPMP is written early in the process, before the *Project Agreement* is finalized. It is reviewed both by management and by developers to ensure that the schedule is feasible and that important risks have not been omitted. The SPMP is then updated throughout the process when design decisions are made or problems are discovered. The SPMP, once published, is baselined and put under configuration management. The revision history section of the SPMP provides a history of changes as a list of changes, including author responsible for the change, date of change, and brief description of the change.

11.5.2 Assigning responsibilities

Management assigns two types of roles to developers, management roles, such as team leader and liaison, and technical roles, such as architect, analyst, or tester. Management assigns these roles to an individual or to a team, depending on the effort requirements and the type of role assigned.

Assigning management roles

Management roles are assigned to individual participants. A role such as team leader cannot be filled effectively if shared by two or more participants. First, a team leader fills a communication function by acting as a liaison between higher level management and developers and thus must be able to communicate a consistent picture of status back and forth. Second, although a team leader makes usually decisions seeking the consensus of developers, he or she occasionally has to impose time-critical decisions. Other management roles, such as liaison between teams also require the same degree of consistency and thus are assigned to an individual.

Management seeks three qualities when assigning management roles: the ability to communicate, the ability to recognize risks early, and the discipline to separate management decisions from technical ones. Such qualities often come from experience in the project setting.

Assigning technical roles

Technical roles, unlike management roles, can be assigned to a team of participants. Complex projects have an architecture team and a testing team composed of only system architects and testers, respectively. Once management has defined the responsibilities of the team, the team leader assigns individual tasks to each team member based on availability or skill. Technical roles do not require the same level of consistency as management roles do given that their tasks are better defined and that they rely on management and liaisons for coordination.

The skills required for each technical role vary significantly with the role. As design methods and implementation technology evolve rapidly, however, there is a greater shortage of technical skills than management skills, in which case, much training is required at the beginning of the project. Assigning technical roles to teams also allows mixing experts with novices, who can then complement their training with the availability of experienced mentors.

Selecting team sizes

Although technical roles can be assigned to teams, the size of a single team is constrained by management and communication overhead. The more participants in one team, the higher the communication overhead and the lower the effectiveness of the team. The observations below are adapted from [Kayser, 1990], who originally applied these heuristics to the formation of subgroups during meetings. Given that team meetings are a significant tool for collecting status, negotiation, and decision making, constraints on the number of participants in meetings are the upper limits on the team size.

- *Three members.* During meetings, every member has a chance to speak. Issues are discussed thoroughly and resolved easily. A possible problem with this team size is that one member can dominate the other two. Also, small teams suffer from the cumulation of multiple roles for each participant (“too many hats” syndrome).
- *Four members.* Starting with teams of this size allows the team to continue to function even if a participant drops out of the team. Reaching consensus with a team of even numbers can be problematic, however, as there can be a tie when voting. As a result, resolving issues can take a relatively long time.
- *Five and six members.* This is the ideal size for a software development team. Members can still meet face to face. Diverse perspectives, supported by a mix of ideas, opinions, and attitudes promotes creative thinking. Single roles can be assigned to each member, creating a miniorganization in which each member complements the others.
- *Seven members.* This is still a relatively effective team size, but team meetings tend to become long. Reviewing the status requires more than half an hour. If teams of seven are necessarily, it is advisable to form subteams during the formal team meetings, each having the task to discuss and resolve a subset of the open issues from the agenda.
- *Eight and more members.* Teams with this size become difficult to manage. The internal structure often starts to break down into subteams. Coalitions and side conversations often occur during formal meetings. Members have more opportunities to compete than to cooperate. Although results are usually satisfactory, the time to reach these results is longer than with a team of smaller size.

11.5.3 Communicating about project management

Communication about project management is difficult for several reasons. As in the case of analysis, management, developers, and the client have different backgrounds and do not discuss the system at the same level of abstraction. Management, developers, and the client each have a vested interest in the system, which creates a barrier during communication. The client strives for a useful system at the lowest cost. Management strives to respect time and budget constraints, either by asking a higher price from the client or by pressuring developers to develop faster in order to lower cost. Developers have the technical knowledge and take interventions by the client or the management as an intrusion in their territory. These factors, already present during analysis, become worse when communication is about management, including the allocation of resources and the review of status.

Below, we discuss heuristics for facilitating communication among all three parties.

Communicating with the client

The communication with the client is characterized by its episodic nature. The client is present at the kickoff, during external reviews, and during the acceptance test. Otherwise, the client is usually not available. This motivates a face-to-face communication for making decisions followed by short, written documents as a record of these decisions. These include the *Problem Statement*, used when kicking off the project, the *Project Agreement*, when the scope of

the project is baselined, and the *Client Acceptance*, once the product is delivered. Detailed status information and partial drafts of technical information are usually not shared with the client.

Communicating with team leaders

The communication between project management and the team leaders determines the quality of the status information available to project management. If project management is seen reacting prematurely to status reports, a team leader will stop reporting the problems he believes he can handle alone. If the project manager believes the team leaders take too much initiative, he will withhold critical information. Setting up a weekly meeting with project management and team leaders, in which the information flows freely, is a critical element for addressing issues in time. The project manager can support this communication by separating status meetings from decision meetings. The project manager can, moreover, reward problem reporting by making available to the reporting team leader the appropriate resources to deal with the problem.

Communicating with developers

The project manager does not usually communicate directly with developers in the same way he meets with the team leaders. The main channel of communication are the SPMP, project reviews, and team leaders. Most often, team leaders act as a shield between project management and the developers filling the gap between the highest and the lowest level of abstraction. The communication between the team leader and developers is critical, as the team leader usually has the technical expertise to assess problems encountered by the developers.

11.6 Exercises

1. What are the relative advantages of flat staffing versus gradual staffing?
2. What is the difference between status meetings and decision meetings? Why should they be kept separate?
3. Why should the role of architect and team leader be assigned to distinct persons?
4. Draw a UML model of the team organization of the OWL project for each the three main phases (i.e., initiation, steady state, termination).
5. Draw a detailed task plan of the system design of the MyTrip system presented in Chapter 6, *System Design*.
6. Estimate the time to complete each task of the task model produced in Exercise 5 and determine the critical path.
7. Identify, prioritize, and plan for the top five risks related to the system design of MyTrip presented in Chapter 6, *System Design*.
8. Identify, prioritize, and plan for the top five risks related to the user interface subsystem of CTC presented in Chapter 8, *Rationale Management*.
9. Linux, developed using the bazaar model, is more reliable and more responsive than many operating systems running on Intel PCs. Discuss in some detail why the bazaar model should, however, not be used for the Space Shuttle control software.

10. Contrast the roles of project manager, development lead, and technical lead, as defined in Section 11.3.2. In particular, describe the tasks each lead is responsible for and which decisions they have final say on.

References

- [Allen, 1985] T. J. Allen, *Managing the Flow of Technology: Technology Transfer and the Dissemination of Technological Information within the R&D Organization*, 2nd ed. MIT Press, Cambridge, MA, 1995.
- [Boehm, 1987] B. Boehm, "A spiral model of software development and enhancement," *Software Engineering Project Management*, pp. 128–142, 1987.
- [Boehm, 1991] B. Boehm, "Software risk management: Principles and practices," *IEEE Software*, vol. 1, pp. 32–41, 1991.
- [Brooks, 1975] F. P. Brooks, *The Mythical Man Month*, Addison-Wesley, Reading, MA, 1975.
- [Bruegge et al., 1997] B. Bruegge, S. Chang, T. Fenton, B. Fernandes, V. Hartkopf, T. Kim, R. Pravia, & A. Sharma, "Turning lightbulbs into objects," Experience paper presented at OOPSLA'97. Atlanta, 1997.
- [Carr et al., 1993] M. J. Carr, S. L. Konda, I. Monarch, F. C. Ulrich, & C. F. Walker, "Taxonomy-based risk identification," Technical Report CMU/SEI-93-TR-6, Software Engineering Institute, Carnegie Mellon Univ., Pittsburgh, PA, 1993.
- [Charette, 1989] R. N. Charette, *Software Engineering Risk Analysis and Management*. McGraw-Hill, New York, 1989.
- [Hartkopf et al., 1997] V. Hartkopf, V. Loftness, A. Mahdavi, S. Lee, & J. Shankavarm, "An integrated approach to design and engineering of intelligent buildings—The Intelligent Workplace at Carnegie Mellon University," *Automation in Construction*, vol. 6, pp. 401–415, 1997.
- [Hauschildt & Gemuenden, 1998] J. Hauschildt & H. G. Gemuenden, *Promotoren*. Gabler, Wiesbaden, Germany, 1998.
- [Hillier & Lieberman, 1967] F. S. Hillier & G. J. Lieberman, *Introduction to Operation Research*. Holden-Day, San Francisco, 1967.
- [IEEE Std. 1058.1-1993] *IEEE Standard for Software Project Management Plans*, IEEE Computer Society, New York, 1993.
- [Kayser, 1990] T. A. Kayser, *Mining Group Gold*. Serif, El Segundo, CA, 1990.
- [Kemerer, 1997] C. F. Kemerer, *Software project management: readings and cases*. Irwin/McGraw-Hill, Boston, MA, 1997.
- [OWL, 1996] *OWL Project Documentation*, School of Computer Science, Carnegie Mellon Univ., Pittsburgh, PA, 1996.
- [Raymond, 1998] E. Raymond, "The Cathedral and the Bazaar," Available at <http://www.tuxedo.org/~esr/writings/cathedral-bazaar/cathedral-bazaar.html>, 1998.
- [Rogers et al., 1986] *The Presidential Commission on the Space Shuttle Challenger Accident Report*. June 6, 1986.
- [Vaughan, 1996] D. Vaughan, *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA*. The University of Chicago, Chicago, 1996.
- [Weinberg, 1971] G. M. Weinberg, *The Psychology of Computer Programming*, Van Nostrand, New York, 1971.